ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 18

Herencia, interfaces y polimorfismo

- El problema: código repetido
- Herencia con extends
- parent:: y sobrescribir métodos
- Clases abstractas y final
- Interfaces: el contrato
- Polimorfismo: la misma orden, distintas respuestas
- Traits: compartir código sin ser familia
- ¿Herencia, interfaz o trait?
- Clases en archivos: el orden importa
- Los errores típicos de hoy



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En la clase 17 aprendisteis a fabricar objetos. Hoy aprendemos a **relacionarlos**. En una academia hay alumnos, profesores y personal de administración: todos tienen nombre y email, pero cada uno tiene lo suyo. ¿Copiamos el mismo código en tres clases? Ni hablar. Para eso están la **herencia** y las **interfaces**.

Y al final llega la palabra más rimbombante del curso: **polimorfismo**. Os prometo que es mucho más sencilla de lo que suena. 😉

1

🤔 El problema: código repetido



2

📘 Herencia con extends



3

👍 parent:: y sobrescribir métodos



4

🔍 Clases abstractas y final



5

👌 Interfaces: el contrato



6

🚀 Polimorfismo: la misma orden, distintas respuestas



7

😮 Traits: compartir código sin ser familia



8

🤔 ¿Herencia, interfaz o trait?



9

⚠️ Clases en archivos: el orden importa



10

🐞 Los errores típicos de hoy



11

🚀 Ejercicios



12

👋 Lo que tenéis que llevaros de hoy





CLASE 18 · PUNTO 1

🤔 El problema: código repetido

Mirad estas dos clases:

01-codigo-repetido.php — php-paso-a-paso/clase18 — Visual Studio Code

01-codigo-repetido.php

```php
<?php
class Alumno
{
    public function __construct(public string $nombre, public string $email, public float
$nota)
    {
    }

    public function saludar(): string
    {
        return "Hola, soy {$this->nombre} ({$this->email})";
    }
}

class Profesor
{
    public function __construct(public string $nombre, public string $email, public string
$especialidad)
    {
    }

    public function saludar(): string
    {
        return "Hola, soy {$this->nombre} ({$this->email})";
    }
}
```

El nombre, el email y `saludar()` están **repetidos**. Si mañana cambia el saludo, hay que tocarlo en dos sitios... o en diez, si hay diez tipos de usuario. Es el mismo problema que resolvimos con las funciones en la clase 10, pero ahora con clases.

📄 Ejemplo: `01-codigo-repetido.php`



CLASE 18 · PUNTO 2

📘 Herencia con extends

La solución es sacar lo común a una clase **padre** y que las demás **hereden** de ella con `extends` [1]:

02-extends.php — php-paso-a-paso/clase18 — Visual Studio Code

02-extends.php

```php
<?php
class Usuario
{
    public function __construct(public string $nombre, public string $email)
    {
    }

    public function saludar(): string
    {
        return "Hola, soy {$this->nombre} ({$this->email})";
    }
}

class Alumno extends Usuario
{
    public float $nota = 0;

    public function haAprobado(): bool
    {
        return $this->nota >= 5;
    }
}

class Profesor extends Usuario
{
    public string $especialidad = "";
}

$marta = new Alumno("Marta", "marta@ejemplo.com");   // usa el constructor del padre
echo $marta->saludar();                               // heredado de Usuario
echo $marta->haAprobado() ? "Aprueba" : "Suspende";  // propio de Alumno
```

Una clase **hija** recibe **todo** lo del padre (propiedades y métodos) y añade lo suyo. El código común está ahora en un solo sitio.

📄 Ejemplo: `02-extends.php`



🎯 **La prueba del «es un»:** la herencia tiene sentido cuando podéis decir «un Alumno **es un** Usuario» o «un Coche **es un** Vehículo». Si la frase suena rara («un Carrito es un Producto»), la herencia no es la herramienta.

Y PHP lo sabe: `$marta instanceof Usuario` devuelve `true`. Un alumno **es** un usuario a todos los efectos.



CLASE 18 · PUNTO 3

👍 parent:: y sobrescribir métodos

¿Y si la hija necesita un dato más en el constructor? Escribe el suyo propio y llama al del padre con `parent::__construct()` para que haga su parte:

03-parent-sobrescribir.php — php-paso-a-paso/clase18 — Visual Studio Code

03-parent-sobrescribir.php

```php
<?php
class Profesor extends Usuario
{
    public function __construct(string $nombre, string $email, public string $especialidad)
    {
        parent::__construct($nombre, $email);
    }

    public function saludar(): string
    {
        return parent::saludar() . " y enseño {$this->especialidad}";
    }
}

$ramon = new Profesor("Ramón", "ramon@alpe.es", "PHP");
echo $ramon->saludar();   // Hola, soy Ramón y enseño PHP
```

Fijaos en `saludar()`: el profesor **sobrescribe** el método del padre. Cuando un profesor saluda, se usa su versión. Y dentro, `parent::saludar()` aprovecha lo que ya hacía el padre y le añade algo, sin repetirlo.

Ahora sí: protected

En la clase 17 os presenté `protected` y os dije que esperaba a la herencia. Aquí está:

Visibilidad	Dentro de la clase	En las clases hijas	Desde fuera
`public`	✅	✅	✅
`protected`	✅	✅	❌
`private`	✅	❌	❌

Si el padre tiene `protected string $email`, el profesor puede usar `$this->email` en sus métodos, pero desde fuera nadie puede escribir `$ramon->email`.

📄 Ejemplo: `03-parent-sobrescribir.php`



CLASE 18 · PUNTO 4

🔍 Clases abstractas y final

¿Tiene sentido crear un `Usuario` a secas, que no sea alumno, ni profesor, ni nada? En nuestra academia, no. Para eso existen las **clases abstractas**: moldes **incompletos** que solo sirven para heredar de ellos [2].

04-abstractas.php — php-paso-a-paso/clase18 — Visual Studio Code

04-abstractas.php

```php
<?php
abstract class Usuario
{
    public function __construct(public string $nombre, public string $email)
    {
    }

    abstract public function obtenerRol(): string;

    public function presentarse(): string
    {
        return "{$this->nombre} · {$this->obtenerRol()}";
    }
}

class Alumno extends Usuario
{
    public function obtenerRol(): string
    {
        return "Alumno";
    }
}

new Usuario("Nadie", "nadie@ejemplo.com");   // ❌ Cannot instantiate abstract class Usuario
```

Un **método abstracto** no tiene código: el padre dice **qué** debe existir y cada hija decide **cómo**. Y si una hija se olvida de escribirlo, PHP no la deja ni existir. Es una forma de que nadie se salte las reglas.

Fijaos en un detalle precioso: `presentarse()` está en el padre y llama a `obtenerRol()`, que todavía no existe en el padre. Funciona porque, cuando se ejecute, siempre será desde un alumno, un profesor... que sí lo tienen.

final: hasta aquí

`final` hace lo contrario: impide heredar.

Escribís	Significa
`final class Administrador`	Nadie puede hacer `extends Administrador`
`final public function calcular()`	Las hijas no pueden sobrescribir este método

📄 Ejemplo: `04-abstractas.php`



Una **interfaz** es un **contrato**: una lista de métodos que una clase se compromete a tener. No lleva código, solo las «firmas» de los métodos [3].

Pensad en un enchufe. A la pared le da igual lo que conectéis: una lámpara, un portátil o una tostadora. Solo exige que la clavija tenga la forma correcta. La interfaz es la forma de la clavija.

05-interfaces.php — php-paso-a-paso/clase18 — Visual Studio Code

05-interfaces.php

```php
<?php
interface Notificador
{
    public function enviar(string $destinatario, string $mensaje): string;
}

class NotificadorEmail implements Notificador
{
    public function enviar(string $destinatario, string $mensaje): string
    {
        return "📧 Email a {$destinatario}: {$mensaje}";
    }
}

class NotificadorSms implements Notificador
{
    public function enviar(string $destinatario, string $mensaje): string
    {
        return "📱 SMS a {$destinatario}: " . mb_substr($mensaje, 0, 30);
    }
}
```

La magia llega al usar la interfaz como **tipo** de un parámetro:

05-interfaces.php — php-paso-a-paso/clase18 — Visual Studio Code

05-interfaces.php

```php
<?php
function avisar(Notificador $notificador, string $destinatario): string
{
    return $notificador->enviar($destinatario, "Mañana no hay clase");
}

echo avisar(new NotificadorEmail(), "marta@ejemplo.com");
echo avisar(new NotificadorSms(), "612345678");
```

`avisar()` no sabe ni le importa si es un email o un SMS. Solo sabe que lo que recibe **cumple el contrato** y, por tanto, tiene `enviar()`. Si mañana añadís `NotificadorTelegram`, `avisar()` no cambia ni una coma.

📄 Ejemplo: `05-interfaces.php`

	Clase abstracta	Interfaz
¿Puede tener código?	Sí, métodos completos y abstractos	No, solo las firmas
¿Puede tener propiedades?	Sí	No (solo constantes)
¿Cuántas a la vez?	Una sola: `extends Usuario`	Varias: `implements Exportable, Notificador`
Palabra clave	`extends`	`implements`



Polimorfismo viene del griego y significa «muchas formas». En programación quiere decir esto: **la misma orden, y cada objeto la cumple a su manera** [4].

Ya lo habéis visto sin daros cuenta: `enviar()` en los notificadores, `obtenerRol()` en los usuarios. Mirad un ejemplo más con figuras geométricas:

06-polimorfismo.php — php-paso-a-paso/clase18 — Visual Studio Code

06-polimorfismo.php

```php
<?php
interface Figura
{
    public function area(): float;
}

class Circulo implements Figura
{
    public function __construct(private float $radio)
    {
    }

    public function area(): float
    {
        return M_PI * $this->radio ** 2;
    }
}

class Rectangulo implements Figura
{
    public function __construct(private float $base, private float $altura)
    {
    }

    public function area(): float
    {
        return $this->base * $this->altura;
    }
}

$figuras = [new Circulo(2), new Rectangulo(4, 2.5)];
$total = 0;
foreach ($figuras as $figura) {
    $total += $figura->area();   // cada figura sabe calcular SU área
}
```

Imaginad el mismo programa sin polimorfismo: un `if` preguntando «¿es un círculo? entonces π·r²; ¿es un rectángulo? entonces base·altura...». Con cada figura nueva, otro `elseif`. Con polimorfismo, la figura nueva trae su propio `area()` y el bucle no se toca.

📄 Ejemplo: `06-polimorfismo.php` (con un triángulo incluido)



💡 En un bucle como el anterior, el orden de programa de siempre se mantiene: primero las clases, después las funciones, la inicialización (`$figuras` , `$total`), el procesamiento (el bucle) y la salida.



CLASE 18 · PUNTO 7

😮 Traits: compartir código sin ser familia

A veces dos clases que **no son familia** necesitan el mismo comportamiento. Un producto y una factura no tienen nada que ver, pero los dos quieren guardar su fecha de creación. ¿Herencia? No: una factura no «es un» producto. Para estos casos existen los **traits** [5]:

07-traits.php — php-paso-a-paso/clase18 — Visual Studio Code

07-traits.php

```php
<?php
trait TieneFechaCreacion
{
    private ?DateTime $creado = null;

    public function marcarCreado(): void
    {
        $this->creado = new DateTime();
    }

    public function fechaCreacion(): string
    {
        if ($this->creado === null) {
            return "sin fecha";
        }
        return $this->creado->format("d/m/Y H:i");
    }
}

class Producto
{
    use TieneFechaCreacion;
}

class Factura
{
    use TieneFechaCreacion;
}
```

Un trait es un trozo de código que PHP **copia y pega** dentro de cada clase que lo usa con `use`. No crea ninguna relación «es un»: solo ahorra repetir.

📄 Ejemplo: `07-traits.php`



💡 Ese `?DateTime` significa «un `DateTime` **o** `null` ». Es un tipo *nullable*; lo veremos con calma en el próximo bloque.



CLASE 18 · PUNTO 8

🤔 ¿Herencia, interfaz o trait?

Esta tabla os resuelve la duda en la mayoría de casos:

Pregunta	Herramienta	Ejemplo
¿Es un tipo **de** otra cosa y comparte su código?	**Herencia** (`extends`)	Un Profesor **es un** Usuario
¿**Sabe hacer** algo que otras clases distintas también saben hacer?	**Interfaz** (`implements`)	Un Email y un SMS **saben** enviar
¿Varias clases sin relación necesitan el **mismo código**?	**Trait** (`use`)	Producto y Factura guardan su fecha



🎯 **Mi consejo:** no construyáis árboles genealógicos de cinco generaciones. Una herencia de uno o dos niveles es fácil de seguir; más allá, cada cambio en un abuelo afecta a toda la familia. Ante la duda, interfaces.



CLASE 18 · PUNTO 9

⚠️ Clases en archivos: el orden importa

Cuando cada clase va en su archivo, como aprendisteis en la clase 17, hay que cargar **primero al padre**:

08-clases-en-archivos.php — php-paso-a-paso/clase18 — Visual Studio Code

08-clases-en-archivos.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/clases/Usuario.php";    // el padre, primero
require_once __DIR__ . "/clases/Alumno.php";
require_once __DIR__ . "/clases/Profesor.php";
```

Si cargáis `Alumno.php` antes que `Usuario.php`, PHP se para con *Class "Usuario" not found*: no puede construir una hija sin conocer al padre.

📄 Ejemplo: `08-clases-en-archivos.php` y la carpeta `clases/`



💡 Estar pendientes del orden de los `require_once` es un rollo, y con treinta clases, una pesadilla. La **autocarga** del próximo bloque lo resuelve: PHP cargará cada clase sola, en el orden correcto.



CLASE 18 · PUNTO 10

🐞 Los errores típicos de hoy

Mensaje	Qué ha pasado	Solución
`Cannot instantiate abstract class Usuario`	Un `new` de una clase abstracta	Cread un objeto de una de sus hijas
`Class Alumno contains 1 abstract method and must therefore be declared abstract or implement the remaining methods`	La hija no ha escrito un método abstracto del padre	Escribid el método que falta (por ejemplo, `obtenerRol()`)
`Class NotificadorSms contains 1 abstract method…`	La clase no cumple todo el contrato de la interfaz	Escribid todos los métodos de la interfaz, con la misma firma
`Declaration of … must be compatible with …`	Al sobrescribir, habéis cambiado los parámetros o el tipo de retorno	Mantened la misma firma que el padre o la interfaz
`Class Admin cannot extend final class Administrador`	Heredar de una clase `final`	No se puede: es la intención de quien la escribió
`Too few arguments to function Usuario::__construct()`	En el constructor de la hija, falta llamar a `parent::__construct()` o faltan datos	Llamad a `parent::__construct(...)` con todos sus datos
`Class "Usuario" not found`	Habéis cargado la hija antes que el padre	`require_once` del padre primero



CLASE 18 · PUNTO 11

🚀 Ejercicios

- ☐ **01 · Vehículos.** Clase abstracta `Vehiculo` (marca, modelo, método abstracto `ruedas()` y `describir()`), y tres hijas: `Coche` (con puertas, que amplía `describir()`), `Moto` y `Camion` (las ruedas

dependen de los ejes). Contad las ruedas de todo el garaje. → `ej01-vehiculos.php`

- ☐ **02 · Nómina.** Clase abstracta `Empleado` con `sueldoMensual()` y tres tipos: fijo (salario anual entre 12), por horas y becario (`final`, con una ayuda fija). Mostrad la nómina en una tabla con el total. → `ej02-nomina.php`
- ☐ **03 · Exportable.** Una interfaz `Exportable` con `cabeceras()` y `aFila()`, que implementen `Producto` y `Alumno`, y una función `aCsv()` que sirva para los dos. → `ej03-exportable.php`
- ☐ **04 · Figuras ordenadas.** Ordenad una lista de figuras por área con `usort()` y el operador nave espacial `<=>`, y contad cuántos círculos hay con `instanceof`. Pista: un `Cuadrado` puede ser un `Rectangulo` con los dos lados iguales. → `ej04-figuras-ordenadas.php`
- ☐ **05 · Descuentos.** Un trait `ConDescuento` con `aplicarDescuento()` y `precioFinal()`, usado por `Producto` y por `Curso`, aunque cada uno calcule su precio de forma distinta. Pista: un trait puede exigir un método abstracto. → `ej05-trait-descuento.php`
- ☐ **06 · La granja.** Clase abstracta `Animal` con el método abstracto `sonido()` y un `presentarse()` común; `Perro`, `Gato` y `Vaca` (que amplía `presentarse()` con `parent::`). → `ej06-animales.php`
- ☐ **07 · Tipos de cuenta.** Una `Cuenta` base y dos hijas: `CuentaCorriente`, que sobrescribe `retirar()` para permitir un descubierto, y `CuentaAhorro`, que añade `aplicarIntereses()`. → `ej07-cuentas.php`
- ☐ **08 · Métodos de pago.** Una interfaz `MetodoPago` (`nombre()`, `comision()`, `admite()`) con `Tarjeta`, `Bizum` (máximo 1.000 €) y `Transferencia`, y una función `cobrar(MetodoPago $metodo, float $importe)` que sirva para todos. → `ej08-pagos.php`
- ☐ **09 · Un historial para todos.** Un trait `ConRegistro` que guarde las acciones con su hora, usado por `Usuario` y por `Pedido`. → `ej09-trait-registro.php`
- ☐ **10 · Vuela, nada o las dos cosas.** Interfaces `Volador` y `Nadador`; un `Pato` que implementa las dos, un `Pez` y un `Avion`. Recorred la lista y, con `instanceof`, mostrad qué sabe hacer cada uno. → `ej10-capacidades.php`
- ☐ **11 · Componentes HTML.** Clase abstracta `Componente` con un método `final` `html()` que envuelve el contenido, y dos hijas, `Boton` y `Alerta`, que solo deciden su `contenido()`. → `ej11-componentes-html.php`



🏆 **Reto:** en el ejemplo de los notificadores, cread un `NotificadorMultiple` que **también** implemente `Notificador`, reciba en su constructor una lista de notificadores y, al llamar a `enviar()`, envíe el mensaje por todos ellos. ¿A que `avisar()` sigue funcionando sin tocarla?



CLASE 18 · PUNTO 12

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Herencia	`class Alumno extends Usuario` : la hija recibe todo lo del padre
«Es un»	Si la frase no suena bien, la herencia no es la herramienta
`parent::`	Llama al constructor o a un método del padre
Sobrescribir	La hija redefine un método del padre con la misma firma
`protected`	Lo ven la clase y sus hijas, nadie más
Abstracta	Molde incompleto: no se puede hacer `new` y obliga a las hijas
`final`	Nadie puede heredar (o sobrescribir)
Interfaz	Un contrato de métodos: `implements` , y se pueden firmar varios
Polimorfismo	La misma orden y cada objeto responde a su manera
Trait	Código compartido entre clases que no son familia: `use`

📚 Fuentes y documentación oficial

1. Manual de PHP: herencia de objetos
2. Manual de PHP: abstracción de clases
3. Manual de PHP: interfaces de objetos
4. Manual de PHP: instanceof (operadores de tipo)
5. Manual de PHP: traits